

Detailed Explanation of Session 11 Topics in C#

In this session, we will cover several important topics in C# programming, including creating shared assemblies, creating custom attributes, using reflection to explore and load assemblies dynamically, and working with files and streams. Each topic will be explained in detail with examples to enhance your understanding.

1. Creating a Shared Assembly

Creating a shared assembly in C# involves several steps, including generating a strong name key, creating the assembly, signing it, and installing it into the Global Assembly Cache (GAC).

Steps to Create a Shared Assembly

1. Generate a Strong Name Key File

- Use the Strong Name Utility (sn.exe) to create a key file.
- Command:

```
bash
VerifyOpen In EditorRunCopy code
1sn -k SharedMathLibraryKey.snk
```

2. Create a Class Library Project

- Open Visual Studio and create a new Class Library project named SharedMathLibrary.
- Example class:

```
csharp
VerifyOpen In EditorRunCopy code
1namespace SharedMathLibrary
2{
3    public class MathOperations
4    {
5        public int Add(int a, int b) => a + b;
6        public int Subtract(int a, int b) => a - b;
7    }
8}
```

3. Sign the Assembly

- In project properties, go to the **Signing** tab, check **Sign the assembly**, and select the key file.

4. Build the Project

- This generates the SharedMathLibrary.dll.

5. Install the Assembly into the GAC

- Use the command:

```
bash
VerifyOpen In EditorRunCopy code
1gacutil -i SharedMathLibrary.dll
```

6. Create a Console Application to Test the Shared Assembly

- Reference the SharedMathLibrary and use it in your application:

```
csharp
VerifyOpen In EditorRunCopy code
1using SharedMathLibrary;
2
3class Program
4{
5    static void Main()
6    {
7        MathOperations mathOps = new MathOperations();
8        Console.WriteLine(mathOps.Add(3, 5));
9        Console.WriteLine(mathOps.Subtract(10, 4));
10    }
11}
```

7. Run the Console Application

- Execute the application to see the results.

Summary

- **Strong Naming:** Ensures unique identification of the assembly.
 - **GAC:** Allows sharing of assemblies across applications.
-

2. Creating Custom Attributes

Custom attributes in C# allow you to add metadata to your code, which can be used for various purposes.

Steps to Create a Custom Attribute

1. Define the Attribute Class

- Derive from `System.Attribute`.
- Use `AttributeUsage` to specify where it can be applied.

2. Declare Constructors and Properties

- Example of a custom color attribute:

```
csharp
VerifyOpen In EditorRunCopy code
1[AttributeUsage(AttributeTargets.Property)]
2public class ColorAttribute : Attribute
3{
4    public ConsoleColor Color { get; }
5    public ColorAttribute(ConsoleColor color = ConsoleColor.Cyan) =>
6    Color = color;
7}
```

3. Apply the Custom Attribute

```
csharp
VerifyOpen In EditorRunCopy code
1public class Dog
2{
```

```

3    [Color(ConsoleColor.Red)]
4    public string Name { get; set; }
5    [Color]
6    public string Breed { get; set; }
7    public int Age { get; set; }
8}

```

4. Use Reflection to Read Attributes

- Example method to display properties with colors:

```

csharp
VerifyOpen In EditorRunCopy code
1public static void Line(Dog dog)
2{
3    // Implementation to display dog properties with colors
4}

```

Conclusion

Custom attributes enhance the metadata of your classes and can be utilized for various functionalities.

3. Using Reflection to Explore an Assembly

Reflection allows you to inspect and interact with types and members at runtime.

Example: Exploring an Assembly

1. Create a Sample Assembly

```

csharp
VerifyOpen In EditorRunCopy code
1namespace MyAssembly
2{
3    public class MyClass
4    {
5        public int X { get; set; }
6        public int Y { get; set; }
7        public int MyMethod(int count, string text) => X + Y;
8    }
9}

```

2. Load the Assembly and Explore It

```

csharp
VerifyOpen In EditorRunCopy code
1Assembly asm = Assembly.LoadFrom(@"C:\Path\To\MyAssembly.dll");
2Type myClassType = asm.GetType("MyAssembly.MyClass");
3// Display properties and methods

```

3. Create an Instance and Invoke Methods

```

csharp
VerifyOpen In EditorRunCopy code
1object myClassInstance = Activator.CreateInstance(myClassType);
2MethodInfo myMethod = myClassType.GetMethod("MyMethod");
3var result = myMethod.Invoke(myClassInstance, new object[] { 5,
"Hello" });

```

Conclusion

Reflection is a powerful tool for dynamic type inspection and method invocation.

4. Using Reflection to Load an Assembly Dynamically

This allows you to load assemblies at runtime and invoke their methods.

Example: Loading an Assembly Dynamically

1. Create a Sample Assembly

```
csharp
VerifyOpen In EditorRunCopy code
1namespace BusinessLogic
2{
3    public class BankAccount
4    {
5        public int GetBalance(string accountNumber) => new
Random().Next(1000, 5000);
6    }
7}
```

2. Load the Assembly and Invoke a Method

```
csharp
VerifyOpen In EditorRunCopy code
1Assembly assembly = Assembly.LoadFrom(@"path\to\BusinessLogic.dll");
2Type bankAccountType = assembly.GetType("BusinessLogic.BankAccount");
3object bankAccountInstance = Activator.CreateInstance(bankAccountType);
4MethodInfo getBalanceMethod = bankAccountType.GetMethod("GetBalance");
5int balance = (int)getBalanceMethod.Invoke(bankAccountInstance, new
object[] { "123456" });
```

Conclusion

Dynamically loading assemblies provides flexibility in application design, especially for plugin architectures.

5. Files I/O and Streams

Working with files and streams is essential for data persistence in applications.

Key Concepts

- **File Handling:** Reading from and writing to files.
- **Streams:** Abstract representations of data sources and destinations.

Example: Reading and Writing Files

1. Writing to a File

```
csharp
VerifyOpen In EditorRunCopy code
1using (StreamWriter writer = new StreamWriter("example.txt"))
2{
3    writer.WriteLine("Hello, World!");
```

```
4}
```

2. Reading from a File

```
csharp
VerifyOpen In EditorRunCopy code
1using (StreamReader reader = new StreamReader("example.txt"))
2{
3    string content = reader.ReadToEnd();
4    Console.WriteLine(content);
5}
```

Conclusion

Understanding file I/O and streams is crucial for effective data management in applications.

By following these detailed explanations and examples, you should have a comprehensive understanding of the topics covered in Session 11. If you have any further questions or need clarification on any point, feel free to ask!